

Assignment: Architecting and Implementing a Resilient Global Telemetry Platform

Context:

You are the lead data engineer for a global logistics company. Your fleet of 500,000 vehicles streams telemetry data (engine heat, speed, location, battery efficiency) 24/7. You are tasked with building a scalable platform to process this massive influx of data for both real-time monitoring and historical predictive maintenance.

Please refer to the "Big Data Platforms & Analytics [Handout and COD].xlsx" document for foundational concepts while completing this assignment.

Part 1: System Architecture & Data Paradigms (Modules 1 & 2)

1. **Scaling Strategy:** Explain the physical hardware limitations of single-node systems (the "Wall" in Hardware). Justify why horizontal scaling (Scale-Out) is required for this telemetry platform over vertical scaling. Ensure you address the core architectural constraints—the Three Vs of Big Data—in your justification.
2. **Consistency Models:** For the high-velocity ingestion of vehicle coordinates, contrast the ACID and BASE consistency models. Using the CAP Theorem, explain which model you would choose for this specific workload and why.

Part 2: Batch Processing & MapReduce (Modules 3 & 4)

1. **Logical Flow:** Design a MapReduce logical data flow to aggregate the total miles driven per vehicle model. Clearly map out the Split, Map, Shuffle, Sort, and Reduce phases.
2. **Hadoop vs. Spark:** Iterative machine learning algorithms will eventually run on this data. Explain why Spark's in-memory computing model overcomes Hadoop's disk-bound I/O bottlenecks for these iterative algorithms.

Part 3: PySpark Implementation & Resilience (Modules 5 & 8)

Write the PySpark code to process a historical batch dataset of this telemetry data. Your code and accompanying explanation must address the following:

1. **Transformations and Actions:** Ingest the data and calculate the average engine temperature per vehicle model. Identify which parts of your code represent narrow dependencies (no shuffle) and which represent wide dependencies (forcing a network shuffle).
2. **Optimization:** Some specific delivery trucks generate 1000x more logs than others, causing severe data skew. Implement a **salting strategy** in your PySpark code to mitigate this skew during aggregation. Define your **partitioning strategy** (e.g., Hash vs. Range partitioning) to optimize the shuffle phase.

3. **Fault Tolerance:** Distributed memory is volatile. Explain how Spark uses Resilient Distributed Datasets (RDDs) and lineage graphs to maintain fault tolerance without heavy data replication.
4. **Checkpointing:** In your code, simulate a scenario where the lineage depth grows excessively long due to iterative processing. Implement a strategic **checkpointing mechanism** to truncate the DAG, and explain how this prevents StackOverflow errors and stabilizes recovery time.

Part 4: Advanced Execution Mechanics & Resilience Strategies (Modules 5-8)

To complete the pipeline, architect the underlying execution and resilience strategies for your Spark application:

1. **The Execution Model & DAGs (Module 6):** Spark does not execute your transformations immediately. Explain the performance mechanics of **Lazy Evaluation**. As your telemetry data is processed, map out how the Spark engine constructs a Directed Acyclic Graph (DAG) and explain exactly how it uses **Wide Dependencies** to decompose the logical plan into physical execution **Stages**.
2. **Data Locality & Fault Tolerance (Modules 5 & 7):** Moving massive amounts of telemetry data across a network is incredibly slow. Explain Spark's "Don't move data, move code" philosophy and how the principle of **Data Locality** minimizes network congestion. Furthermore, contrast Spark's **Lineage-based recovery** with Hadoop's data replication strategy—how does Spark maintain stability without the heavy I/O bandwidth tax of copying data?
3. **Mitigating Lineage Liability (Module 8):** Assume you are running a highly iterative calculation on the telemetry data that updates its state hundreds of times. Explain the **"Liability of Lineage"** in this scenario, specifically addressing the risks of StackOverflow errors and degraded recovery times. Detail how implementing a **Checkpointing** strategy breaks the family tree (truncates the DAG) to solve this, and strictly differentiate how checkpointing behaves compared to standard Spark caching.